第22课 贪心算法入门

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：找零钱（最少硬币）

题目描述：

有四种硬币，面额分别为 10 元、5 元、2 元、1 元，每种数量无限。输入一个正整数 n（1 ≤ n ≤ 1000），输出凑出 n 元最少需要多少个硬币，以及每种硬币用了多少个。

输入样例：

```
36
```

输出样例：

```
10元硬币:3个
5元硬币:1个
2元硬币:0个
1元硬币:1个
最少需要5个硬币
```

讲解思路：

1. 把面额从大到小排好：10、5、2、1。
2. 从最大面额开始，尽量多用：`个数 = 剩余金额 / 面额`，然后 `剩余金额 = 剩余金额 % 面额`。
3. 循环处理每个面额，最后输出结果。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
 int n;
 cin >> n;

 int coins[] = {10, 5, 2, 1}; // 面额从大到小
 int count[4] = {0}; // 记录每种硬币用了几个
 int total = 0; // 总硬币数
 int remain = n; // 剩余金额

 for (int i = 0; i < 4; i++) {
 count[i] = remain / coins[i]; // 尽量多用当前面额
 remain = remain % coins[i]; // 更新剩余金额
 total = total + count[i]; // 累加硬币数
 }

 // 输出结果
 for (int i = 0; i < 4; i++) {
 cout << coins[i] << "元硬币:" << count[i] << "个" << endl;
 }
 cout << "最少需要" << total << "个硬币" << endl;

 return 0;
}
```

代码逐行讲解：

- `int coins[] = {10, 5, 2, 1};` 面额数组，**从大到小**排列，这是贪心的关键！
- `count[i] = remain / coins[i];` 整除，算出当前面额最多能用几个。
- `remain = remain % coins[i];` 取余，算出剩下还需要凑多少。
- 整个循环就是"先用大的，用不了再用小的"——这就是贪心！

例题 2：活动安排（简化版）

题目描述：

小明周末想参加尽可能多的兴趣班。有 n 个兴趣班（1 ≤ n ≤ 100），每个兴趣班有开始时间和结束时间（用整数表示）。同一时间只能参加一个兴趣班。如果一个兴趣班在时间 t 结束，另一个在时间 t 开始，可以都参加（来得及赶过去）。

输入第一行为 n，接下来 n 行每行两个整数表示开始时间和结束时间。输出最多能参加几个兴趣班。

输入样例：

```
5
8 10
9 11
10 12
11 13
8 9
```

输出样例：

```
3
```

讲解思路：

1. **关键策略：按结束时间从小到大排序。**
2. 选第一个活动（结束最早的）。
3. 从第二个活动开始，如果它的开始时间 ≥ 上一个选中活动的结束时间，就选它。
4. 统计选了几个。

> 为什么按结束时间排？因为越早结束，留给后面的时间越多，能安排的活动就越多。

完整代码：

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Activity {
 int start; // 开始时间
 int end; // 结束时间
};

// 比较函数：按结束时间从小到大排序
bool cmp(Activity a, Activity b) {
 return a.end < b.end;
}

int main() {
 int n;
 cin >> n;

 Activity a[105];
 for (int i = 0; i < n; i++) {
 cin >> a[i].start >> a[i].end;
 }

 // 第一步：按结束时间排序（贪心的预处理）
 sort(a, a + n, cmp);

 // 第二步：贪心选择
 int count = 1; // 第一个活动一定选
 int lastEnd = a[0].end; // 记录上一个选中活动的结束时间

 for (int i = 1; i < n; i++) {
 if (a[i].start >= lastEnd) { // 不冲突，可以选
 count++;
 lastEnd = a[i].end; // 更新结束时间
 }
 }

 cout << count << endl;

 return 0;
}
```

代码逐行讲解：

- `struct Activity` 结构体，把开始时间和结束时间"打包"在一起。（如果学生还没学结构体，可以用两个数组 `s[]` 和 `e[]` 代替，见下方备用版本。）
- `bool cmp(...)` 自定义比较函数，告诉 `sort` 按结束时间排序。
- `sort(a, a + n, cmp);` 上节课学的 `sort`，这里派上大用场了！
- `if (a[i].start >= lastEnd)` 判断新活动和上一个选中的活动有没有时间冲突。

备用版本（不用结构体，用两个数组）：

```
#include <iostream>
#include <algorithm>
using namespace std;

int s[105], e[105], order[105]; // 开始时间、结束时间、排序用的下标

bool cmp(int i, int j) {
 return e[i] < e[j]; // 按结束时间排
}

int main() {
 int n;
 cin >> n;

 for (int i = 0; i < n; i++) {
 cin >> s[i] >> e[i];
 order[i] = i;
 }

 sort(order, order + n, cmp); // 对下标排序

 int count = 1;
 int lastEnd = e[order[0]];

 for (int i = 1; i < n; i++) {
 int idx = order[i];
 if (s[idx] >= lastEnd) {
 count++;
 lastEnd = e[idx];
 }
 }

 cout << count << endl;
 return 0;
}
```

附：例题源代码与运行结果

例题 1：Making Change (Minimum Coins)

```
// Given coin denominations 10, 5, 2, 1 (unlimited supply),
// find the minimum number of coins to make n yuan.
#include <iostream>
using namespace std;

int main() {
 int n;
 cin >> n;

 int coins[] = {10, 5, 2, 1}; // Denominations from large to small
 int count[4] = {0}; // Number of each coin used
 int total = 0; // Total coin count
 int remain = n; // Remaining amount

 for (int i = 0; i < 4; i++) {
 count[i] = remain / coins[i]; // Use as many large coins as possible
 remain = remain % coins[i]; // Update remaining amount
 total = total + count[i]; // Accumulate total count
 }

 // Print result
 for (int i = 0; i < 4; i++) {
 cout << coins[i] << " yuan coins: " << count[i] << endl;
 }
 cout << "Minimum coins needed: " << total << endl;

 return 0;
}
```

Sample Input

```
36
```

Expected Output

```
10 yuan coins: 3
5 yuan coins: 1
2 yuan coins: 0
1 yuan coins: 1
Minimum coins needed: 5
```

例题 2：Activity Selection (Simplified)

```
// Given n activities with start and end times, find the maximum number
// of non-overlapping activities. An activity can start right when another ends.
/*
#include <iostream>
#include <algorithm>
using namespace std;

struct Activity {
 int start; // Start time
 int end; // End time
};

// Comparison function: sort by end time ascending
bool cmp(Activity a, Activity b) {
 return a.end < b.end;
}

int main() {
 int n;
 cin >> n;

 Activity a[105];
 for (int i = 0; i < n; i++) {
 cin >> a[i].start >> a[i].end;
 }

 // Step 1: Sort by end time (greedy preprocessing)
 sort(a, a + n, cmp);

 // Step 2: Greedy selection
 int count = 1; // Always pick the first activity
 int lastEnd = a[0].end; // Track end time of last selected activity

 for (int i = 1; i < n; i++) {
 if (a[i].start >= lastEnd) { // No conflict, select it
 count++;
 lastEnd = a[i].end; // Update end time
 }
 }

 cout << count << endl;

 return 0;
}
*/
```

Sample Input

```
5
8 10
9 11
10 12
11 13
8 9
```

Expected Output

```
3
```